NEARCASH · THE 3D-PRINTED ATM

Assembly instructions

From the printed parts to the first banknote: what you need, what to measure, how to prepare the servos, how to put the ATM together in nine steps, and how to calibrate it.

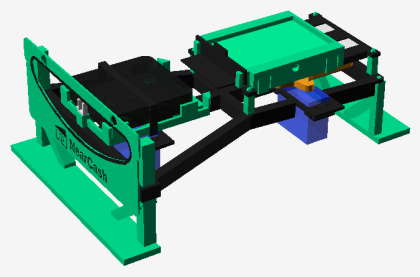


1. How it works
2. What you need
3. Before printing everything
4. Preparing the servos
5. Assembly, in nine steps
6. Motion and calibration
7. If something goes wrong

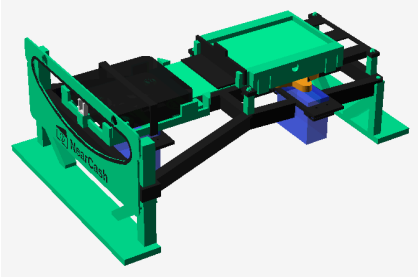
Made from `hardware/atm/nearcash-atm.scad` by `make-instructions.py`. The pictures are the model itself: if you change a measurement, build this PDF again.

1 · How it works

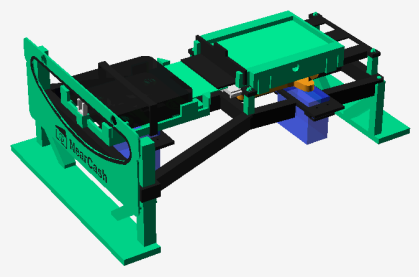
Blank cards wait in the hopper. When a sale is paid, a carriage under the channel floor, moved by a lever on a servo, catches the bottom card with a 0.6 mm finger (thinner than a card, so it never takes two), pushes it under the gate and all the way into the reader. The Raspberry Pi turns the card into a banknote through the reader. The carriage goes back, the turret turns 180° carrying the reader, and the banknote sticks out of the smile: the customer pulls it out of the reader, as at any ATM. When the reader reports the card gone, the turret turns back.



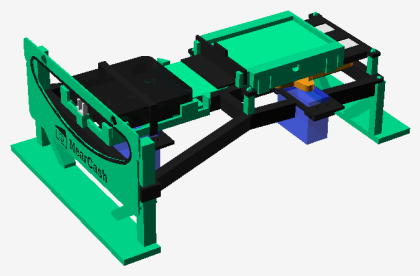
1 Ready: blank cards wait in the hopper; the reader’s slot faces them.



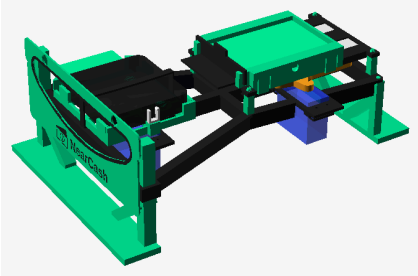
2 The pusher, under the floor, catches only the bottom card and pushes it under the gate.



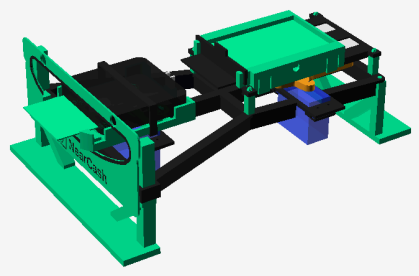
3 The card goes all the way into the reader: the Raspberry Pi issues the banknote.



4 The pusher goes back home. The card stays in the reader, its end sticking out.



5 The turret turns, carrying the reader with the card inside.



6 At 180° the banknote sticks out of the smile: the customer pulls it out.

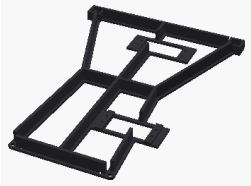
The ATM never pulls the card out itself: the ACR39U is a contact reader with a pocket slot, and printed parts can’t grip a card firmly enough to drag it out. If a card fails its check, the turret doesn’t turn: the card stays in the reader and the ATM is out of service until the operator takes it out.

2 · What you need

PART	DETAILS
Card reader	ACS ACR39U-U1 (contact, USB)
Servos	2 × MG996R (180°), with their cross horns, horn screws and mounting screws (4 per servo, in its bag)
Servo power	4 AA batteries in a holder (NiMH rechargeables: 4.8 V), or a 5 V supply of at least 3 A. It powers the servos only, never through the Pi
Wiring	A small breadboard, 4 male–male and 3 male–female jumper wires: the servos' signals go straight to the Pi's pins, with no driver board
Controller	Raspberry Pi 4 or 5 with 64-bit Raspberry Pi OS and its own power supply; it sits next to the ATM
Cards	Blank NXP J3R180 cards (85.6 × 54 × 0.76 mm), with the applet installed
Screen	An iPhone 17 Pro without a case, and a charging cable with a right-angle plug
Filament	PETG Basic, about 223 g with the test pieces: 106 g green, 117 g black
Tools	A small Phillips screwdriver (the servo and horn screws), a caliper, double-sided tape, a craft knife or fine sandpaper for burrs

No glue and no metal in the structure: the parts fit with printed pegs and rails. The only screws are the servos' own: four hold each servo to the chassis, one holds each horn.

The printed parts



Chassis
1 × · black · plate 8
`chassis`

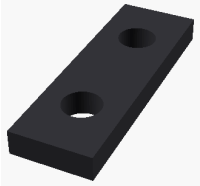
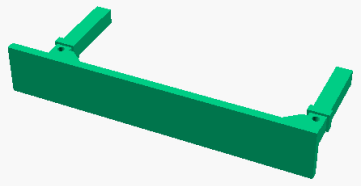
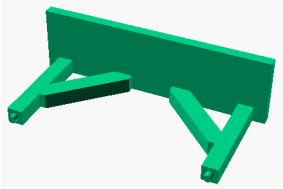


Plate under a servo tab
4 × · black · plate 6
`servo_clamp`



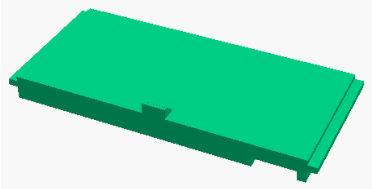
Front legs (the tall posts)
1 × · green · plate 5
`legs_front`



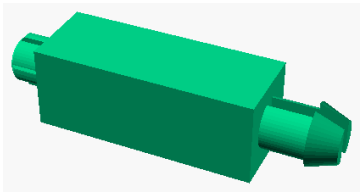
Back legs (the short posts)
1 × · green · plate 5
`legs_back`



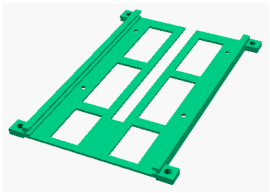
Lever
1 × · black · plate 3
`lever`



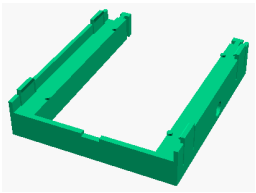
Carriage
1 × · green · plate 7
`carriage`



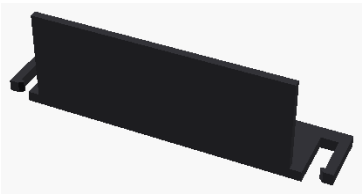
Channel post
4 × · green · plate 10
`channel_post`



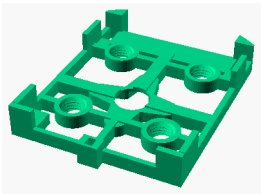
Channel floor
1 × · green · plate 7
`channel_floor`



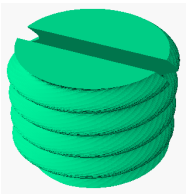
Hopper
1 × · green · plate 11
`hopper`



Gate with its tongue (1.0 mm gap)
1 × · black · plate 12
`gate_1_0_tongue`



Turret
1 × · green · plate 13
`turret`



Reader’s height screw
4 × · green · plate 13
`reader_screw`



Reader’s shim, 0.5 mm (1 notch)
1 × · green · plate 13
`reader_shim_0_5`



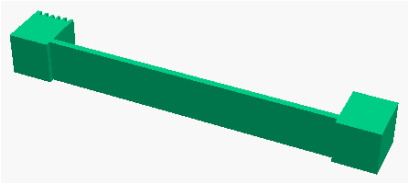
Reader’s shim, 1 mm (2 notches)
1 × · green · plate 13
`reader_shim_1_0`



Reader’s shim, 2 mm (3 notches)
1 × · green · plate 13
`reader_shim_2_0`



Reader’s shim, 4 mm (4 notches)
1 × · green · plate 13
`reader_shim_4_0`



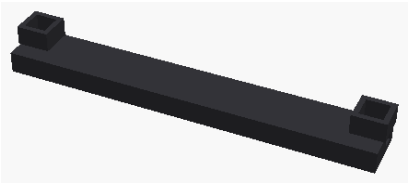
Reader’s shim, 8 mm (5 notches)
1 × · green · plate 13
`reader_shim_8_0`



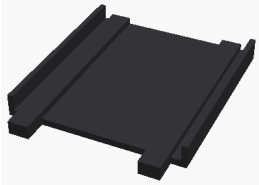
Reader’s strap
1 × · black · plate 3
`turret_strap`



Face (the smile)
1 × · green and black · plate 14
`face`



Phone shelf
1 × · black · plate 9
`phone_shelf`



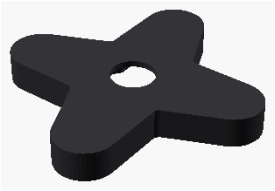
Phone back
1 × · black · plate 9
`phone_back`



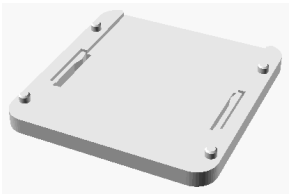
Fit test piece
1 × · black · plate 1
`test_fit`



Reader test piece
1 × · black · plate 1
`test_reader`



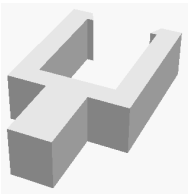
Horn pocket test piece
1 × · black · plate 4
`test_horn`



Dummy reader (until the ACR39U arrives)
1 × · white · plate 15
`dummy_reader`



Dummy reader’s lid
1 × · white · plate 15
`dummy_reader_lid`



Reader’s cable clip
1 × · white · plate 16
`reader_clip`



Reader’s long cable clip (5 mm of shims and more)
1 × · white · plate 16
`reader_clip_long`

`gate_0_9`, `gate_1_0` and `gate_1_2` are spare gates with other gaps (the number is engraved on the hopper side), on plate 2: print them only if 1.1 doesn’t work.

3 · Before printing everything

The model is parametric: everything follows the values at the top of `nearcash-atm.scad`, and the defaults are estimates. Measure your hardware with a caliper. The ATM was laid out for a first estimate of the reader (a card 60 mm in, a slot 6 mm up): what sticks out of the reader swings round with the turret, and the channel, the hopper, the smile and the front posts stand round that circle. So the reader's values move only the turret: `card_insert_depth` sets where the reader sits on it, `reader_slot_bottom` its height, on four screws that also make up any error.

VALUE	HOW TO MEASURE IT	DEFAULT
`card_insert_depth`	Push a card fully into the reader and measure what sticks out. The depth is 85.6 minus that. It sets where the reader sits on the turret.	52.5 mm
`reader_slot_bottom`	With the reader flat on the table and a card in it, the height of the card's lower face above the table. The turret's four screws make up an error of up to 0.7 mm.	4 mm
`reader_w, reader_l, reader_h`	The reader: width, length (slot to cable end) and height. Its pocket on the turret leaves 0.3 mm of play.	69 · 72.2 · 14.5 mm
`horn_span, horn_arm, horn_arm_tip, horn_t, horn_boss_d`	The cross horn (its arms taper): tip to tip, arm width 8 mm out and at the tip, thickness, the middle across the corners.	36 · 7.9 · 4.7 · 3.6 · 14.8 mm
`horn_gap`	With the horn on the servo: from the top of the servo's case to the underside of the horn's arms.	2.3 mm
`servo_tab_top`	From the servo's bottom to the top face of its mounting tabs.	29 mm
`phone_width, phone_thickness, phone_height`	The phone (in its case, if it has one).	71.9 · 8.75 · 150 mm

Check, too, that your servos are the size of the MG996R the model expects: a 40.7 × 19.7 mm body, 54.5 mm from tab to tab.

Then print **plates 1 and 4** (about 20 minutes) and try the three test pieces:

- **`test_reader`**: lay the reader flat on the table, put the piece against its slot and slide a card along its rails. The card must go into the slot without catching its edge and without lifting. If it doesn't, correct `reader_slot_bottom` (within 0.7 mm, the turret's screws do it instead).
- **`test_horn`** (plate 4): press a servo horn into the cross pocket: snug, with no play, since that fit alone holds the turret on its servo. If it is too tight or too loose, change `horn_fit`.
- **`test_fit`**: push the peg into its hole (firm); slide the slice of carriage into the length of rail (free, without wobbling). If a fit is too tight or too loose, change `fit` (sliding) or `press` (pegs).

If you changed a value, build everything again and check it:

```
hardware/atm/export.sh                    # STLs and images
hardware/atm/check-clearances.sh          # nothing touches, at any position
python3 hardware/atm/make-3mf.py          # the Bambu Studio project
python3 hardware/atm/make-instructions.py  # these instructions
```

Then open `nearcash-atm.3mf` in Bambu Studio and print **plates 3, 5 to 14 and 16** (about 7 hours in all): every part is already on its plate, in its colour, with the right settings. No part needs supports. The channel floor prints upside down, the face the cards slide on against the bed: the smooth plate is best for it (with glue stick: PETG sticks too hard to bare smooth PEI), the textured one works too.

Plate 14, the face, is the only one in two colours: the lips round the smile and the NearCash name and logo are black, flush in its front, which prints against the bed. The AMS takes both colours to the same nozzle and swaps them in the first three layers only, with a small prime tower (a few minutes and about 2 g more). The project already maps both to that nozzle.

No reader yet? **Plate 15** (about 50 minutes, in white) is a dummy reader to try the mechanism with: the reader's outside, so it sits in the turret under the strap, and its slot, as high and as deep. A card slides in from the turret's shelf, stops where it would in the reader, and two springs at the slot's sides hold it while the turret turns, as the reader's contacts do. The lid pegs onto the base. It reads nothing: run the motion alone (`cycle.py`), or a whole sale with `pnpm atm:demo --dummy-reader`, which hands out the card blank once paid. If the card comes out too easily or is too hard to pull, change `dummy_grip`.

4 · Preparing the servos

The horns go on with the servos at rest, so wire the servos before the assembly. They plug straight into the Pi: their signal goes to two of its pins, their power comes from the batteries.

SERVO WIRE	TURRET (THE FRONT SERVO)	PUSHER (THE MIDDLE SERVO, WITH THE LEVER)
Orange (signal)	GPIO 12 (pin 32)	GPIO 13 (pin 33)
Red (+)	the batteries' +	the batteries' +
Brown (–)	the batteries' –	the batteries' –

> The batteries' – goes to a GND pin of the Pi too (pin 34): without it the servos can't read the signal. The batteries' + must never touch a pin of the Pi, and the servos never take power from the Pi's 5 V.

The pins are numbered from pin 1, the corner of the 40-pin header away from the USB ports. Pins 32 and 34 are on the row along the board's edge, pin 33 on the inner row, across from 34. On the breadboard, the batteries go on the two rails, each servo's red and brown wires go on them with male–male jumpers, and the orange wires and the rail's – go to the Pi with male–female ones. A 470–1000 µF capacitor (at least 10 V) across the rails is optional: it smooths the dip when the servos start.

On the Pi, once: give pins 12 and 13 to the Pi's hardware PWM, which makes steady pulses, then reboot (`pnpm atm:demo` adds the same line when it prepares the Pi). The 3.5 mm audio jack shares that PWM: nothing may play through it while the servos run.

```
printf '[all]\ndtoverlay=pwm-2chan,pin=12,func=4,pin2=13,func2=4\n' | sudo tee -a
/boot/firmware/config.txt
sudo reboot
```

Then, from the repository on the Pi:

```
pnpm atm:servos                    # both to rest (0°): the pusher first, then the turret; held till
Enter
pnpm atm:servos --pusher-reversed  # the same, for a pusher whose rest is 180°
pnpm atm:servos 13 90              # one servo to an angle: GPIO 12 is the turret, 13 the pusher
```

The script is `hardware/atm/servo.py`. With the hardware PWM the servos hold still; without it the script says so and times the pulses in software (gpiozero): the servos twitch and miss small steps. Every move is calm, from where the servo was left: the scripts write it down on the Pi (`~/.nearcash-atm-servos.json`), since a servo can't tell where it is. When it ends, the servos go limp where they are.

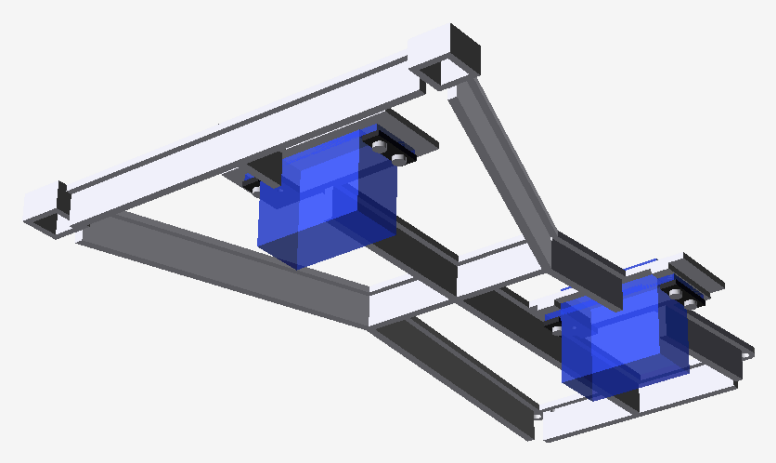
The pusher's direction. Run `pnpm atm:servos 13 0`, then `pnpm atm:servos 13 90`, and watch the shaft from above. If the shaft turns anticlockwise, its rest position is 0°. If it turns clockwise, its rest position is 180°: write it down, because the software will have to reverse this servo (angle → 180° − angle). The turret's direction doesn't matter: its rest is 0°.

The horns. With each servo at rest, held there by `pnpm atm:servos`, fit its cross horn with the screw, its arms as close as possible to the servo's body (one along it, one across). The splines allow only some positions: take the nearest, the small error is corrected in the calibration. The screw's head must not stand more than 1.5 mm proud of the horn: above it sit the reader and the carriage.

> Once the horns are on, don't turn the shafts by hand. If they moved, send the servo back to rest before steps 3 and 6, where you press on the lever and the turret.

5 · Assembly, in nine steps

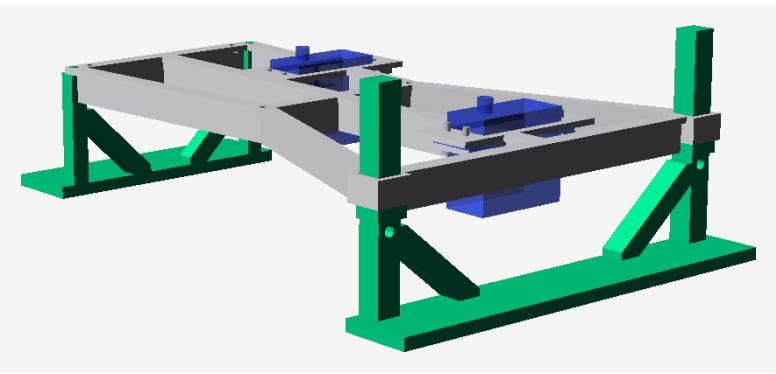
In every picture, the parts mounted in earlier steps are grey and the step's new parts are in their colour. The servos are blue, the reader and the phone dark.



1 The servos in the chassis (seen from below)

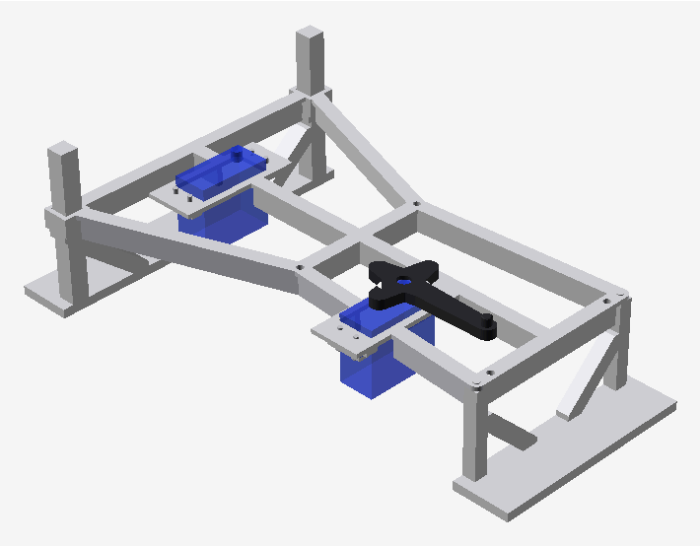
1. Turn the chassis upside down.
2. Push each servo up into its hole from below, shaft up. The shaft goes at the end of the hole **nearest the ATM's centre line** (on the left, seen from the front); the servo's body runs to the right. The front servo turns the turret, the middle one the lever.
3. Push it until its tabs touch the chassis (the middle servo rests on two spacers).
4. Lay a plate (`servo_clamp`) on each tab and screw the servo on: four screws from the servo's bag, through the plate and the slot in the tab, into the holes in the chassis. They cut their own thread: tighten them until the tab is held firm, no further.
5. Leave the servos' cables free: they go to the breadboard and the Pi.

> The plates are there because the slots in the tabs are as wide as the rubber grommets they are made for: a screw head could slip through. Leave the grommets and the brass eyelets out.



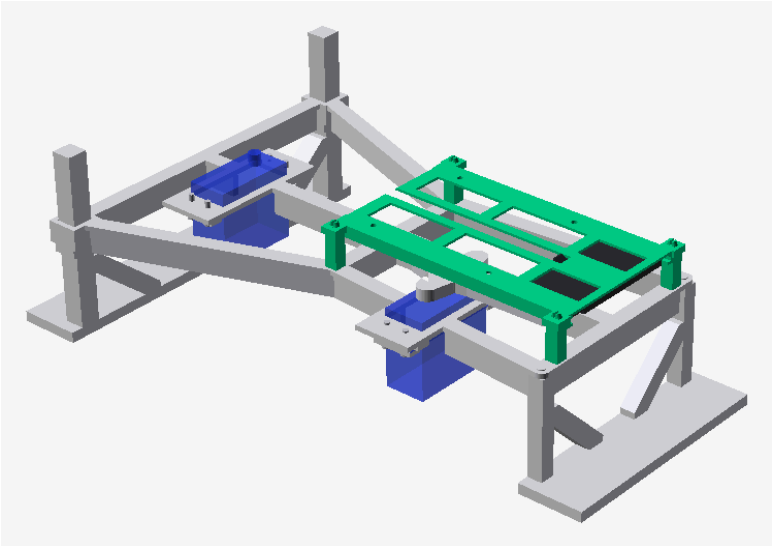
2 The legs

1. Back legs (`legs_back`): press their two pegs into the holes under the chassis, at the back, the base reaching backwards.
2. Front legs (`legs_front`): push their two tall posts up through the front sleeves, the base reaching forwards (the holes for the face's pegs face forward too), until the sleeves rest on the collars.
3. Stand the chassis up: it now stands on the two legs' bases (the middle servo stays 22 mm above the table).



3 The lever

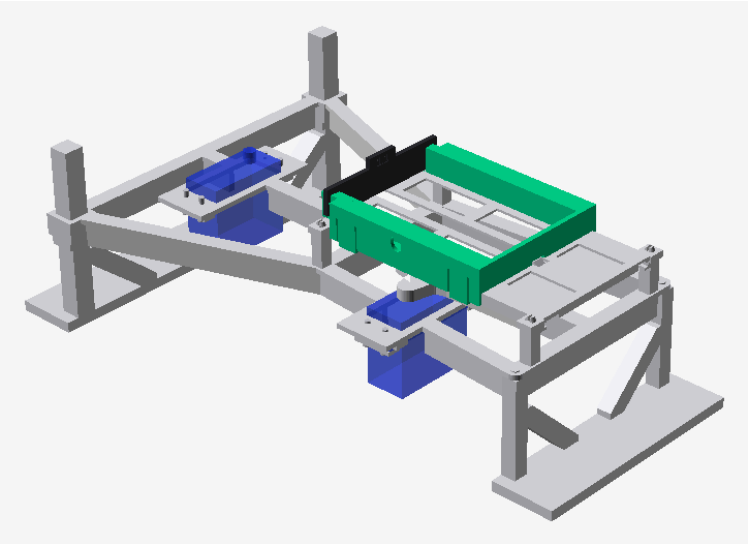
1. Send the middle servo to rest and leave it holding: `pnpm atm:servos 13 0` (or `13 180`, see “Preparing the servos”).
2. Press the lever onto the middle servo’s horn, pin up, arm pointing to the back.
3. Check its direction: going from rest to the other end of its travel, the lever must turn anticlockwise seen from above, towards the middle of the ATM, and end up pointing forward. Then send it back to rest.



4 The channel

1. Slide the carriage into the rails under the channel floor **from the front** (the end towards the turret), finger up in the slot and towards the front: the slot is open only at that end. It must slide freely all the way.
2. Push the four channel posts (`channel_post`) into the holes on top of the chassis, the ribbed peg down: push firmly, the ribs grip the hole.
3. Set the floor on them, guiding the lever’s pin into the slot under the carriage, and press it down over each post until its split head clicks over the floor. To take the floor off, squeeze each head’s two halves together.
4. Test: move the pusher to the other end in steps of 45°, and back. The carriage must follow without sticking.

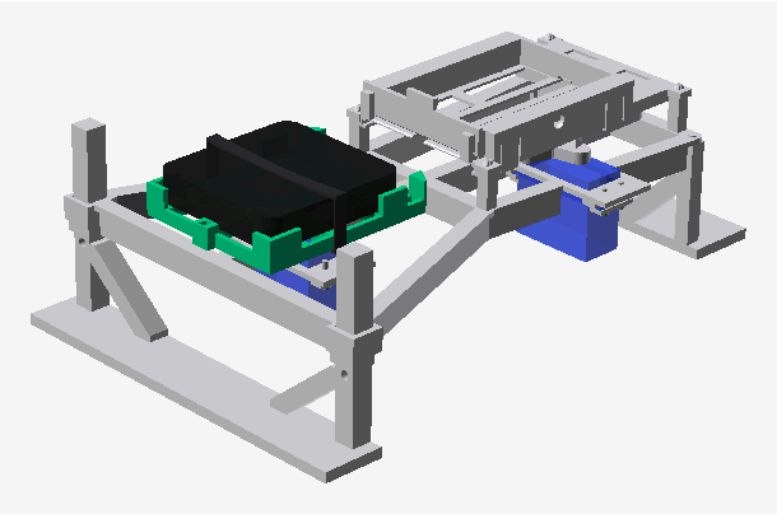
> PETG strings: before sliding the carriage in, clear any wisps of filament out of the rails with a craft knife. A little dry PTFE lubricant on the rails helps: the lever pushes the carriage off its centre line, and too much friction could lock it in its rails, like a drawer pulled by one corner.



5 The hopper and the gate

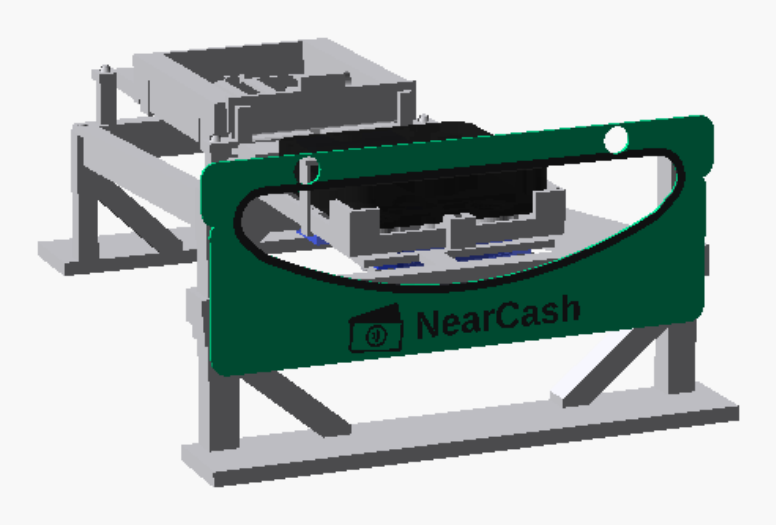
1. Press the hopper onto the floor: its four pegs into their holes, its outer walls down the floor’s sides, just behind the front pads, until the four fingers click under the rails.
2. Slide the gate down the slots in the hopper’s front pillars, its engraved number towards the hopper (the back) and its tongue towards the turret, until its two arms click under the steps on the pillars. The tongue keeps the card flat to the end of the stroke, where the reader resists most.
3. Test with 2–3 cards: move the pusher slowly to the other end and back. Exactly one card must come out.

> If the card stops under the gate, use the spare gate 1.1 or 1.2 (on plate 2, without a tongue). To change the gate, spread its two arms and pull it out by the tab. To take the hopper off, ease its fingers out from under the rails, one side at a time.



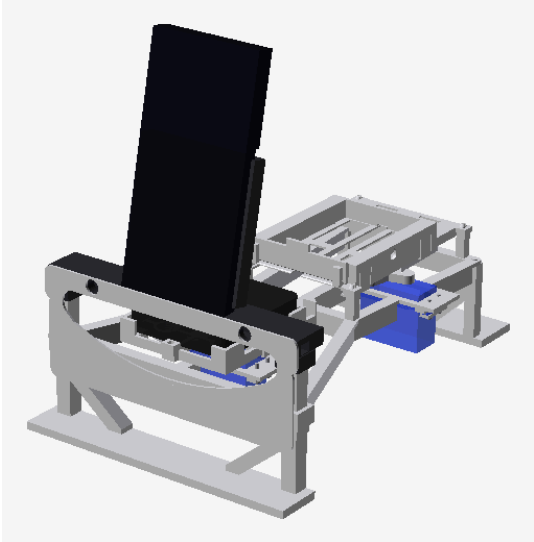
6 The turret and the reader

1. Send the front servo to 0° and leave it holding: `pnpm atm:servos 12 0`.
2. Screw the four height screws (`reader_screw`) into the turret, slot up, until each one's bottom is flush with the turret's underside, then turn each back half a turn: the height for a 4 mm slot.
3. Lay the ACR39U in the turret, its slot towards the hopper, its cable towards the front, its four feet on the screws.
4. Slide a card from the channel, flat on its floor and under the gate's tongue, into the slot: it must go in without catching. Catching under the slot: the reader is too high; above it: too low. Lift the reader and turn all four screws the same way (anticlockwise raises it, about 0.4 mm a quarter turn; never past flush underneath).
5. Snap the reader's strap (`turret_strap`) over it: spread its legs a little, the hooks go under the turret. Its arch presses the reader down at any height.
6. Press the turret onto the front servo's horn, the reader's slot towards the hopper.
7. Press the cable clip's stem (`reader_clip`, white) into the socket behind the reader's cable end, then push the cable's boot down into its fork until it snaps under the lips: the cable's pull then goes to the turret, not the reader.
8. Leave the USB cable a generous loop: its boot reaches into the smile, and the turret turns half a turn.
9. If the carriage leaves the card short of the reader's end (pushed by hand it goes further), push the reader towards its slot and drop a shim (`reader_shim_…`, 1 to 5 notches) in behind it, against the cable-end stops. Start with 0.5 mm and go up half a millimetre at a time (0.5, 1, 1 + 0.5, 2, …: they stack, up to 9 mm); keep the first that lets the card go all the way in. From 5 mm of shims the boot has moved off the clip: put the long clip (`reader_clip_long`) in the same socket instead.
10. Test: 0° → 180° → 0° in steps of 30°, watching the cable. Nothing must touch, and the cable must follow freely.



7 The face

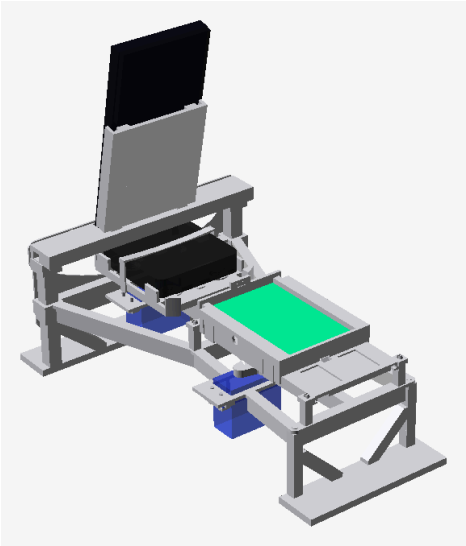
1. Push the face's two pegs into the holes of the tall posts.



8 The phone

1. Press the shelf onto the tall posts' tops (it passes over the turret).
2. Slide the back's two rails into the shelf's sockets.
3. Put the right-angle plug in its pocket and run the cable out along its channel, to the right. The plug's head has about 4.4 mm under the phone: check yours.
4. Stand the phone in the groove, leaning on the back.

> The phone sits high: a hard push at its top can tip the ATM over. Tape the four feet to the counter, or put a weight on the chassis behind the hopper.



9 The cards

1. Load the blank cards (up to about 12) with the chip end towards the gate: that end goes into the reader first.
2. The chip must face the way the reader takes it: try one card by hand first.

> Keep fingers out of the smile while the turret turns.

6 · Motion and calibration

With the reader plugged into the Pi over USB, `pnpm atm:reader` lists the readers and checks the card in it. The ATM server doesn't drive the servos yet (its dispenser is simulated): this is the sequence it will have to run, with the Pi's hardware PWM on the same two pins, and you can try it by hand with `pnpm atm:servos PIN ANGLE`.

STEP	TURRET (GPIO 12)	PUSHER (GPIO 13)	WAITS FOR
Idle	0° (slot to the hopper)	rest (home)	a paid sale
Load	0°	rest → other end, slowly	the reader: card present
Back	0°	other end → rest	
Issue	0°	rest	the banknote issued and checked
Hand out	0° → 180°	rest	the reader: card removed
Ready	180° → 0°	rest	

> The pusher must be home before the turret turns: at full stroke its finger is inside the circle the banknote sweeps.

Calibration. Find the right angle at both ends of each servo, in small steps:

- Turret, loading angle: the reader's slot square to the channel, so the card goes in straight. The horn's splines leave the turret a few degrees off at 0°: run `pnpm atm:servos 12 0` and type `+1` (and Enter) until the slot faces the channel; the angle it ends on is `--turret-in`. If the slot lines up only below 0°, refit the horn one tooth the other way and try again.
- Turret, hand-out angle: the banknote straight out of the smile, found the same way from 180 down (`--turret-out`). The servo turns 180° at most, so with a loading angle above 0° the turret turns a little less than half a turn: up to 10° askew the banknote still clears the smile.
- Pusher at full stroke: the card goes fully into the reader without pushing on its end stop. Start 20° short of the end and go 5° at a time.

The whole cycle. To test the mechanics, `pnpm atm:cycle` runs this sequence on the servos, with pauses instead of the reader and the issuing: a card goes into the reader, the pusher goes home, the turret turns, you pull the banknote out of the smile (you have 8 seconds), and the turret turns back. Put the reader in the turret first, strapped down (without it the card has no slot to go into, and the turret doesn't carry it round), and a few blank cards in the hopper. It starts by bringing the servos to rest, slowly from where they were left, and ends every cycle at rest: the pusher home, the turret at its loading angle, always the pusher first. Ctrl+C stops it and brings them back to rest; Ctrl+C a second time lets them go limp at once, where they are, to free a jammed card by hand.

```
pnpm atm:cycle --dry-run                    # prints the steps, moves nothing
pnpm atm:cycle --stroke 160                 # one cycle, the pusher 20° short of its end
pnpm atm:cycle --stroke 175 --cycles 10     # ten in a row, once the stroke is right
pnpm atm:cycle --pusher-reversed            # for a pusher whose rest is 180°
```

The angles found in the calibration go in as `--stroke`, `--turret-in` and `--turret-out` (by default 180, 0 and 180); `--issue-seconds` and `--take-seconds` change the pauses. The script is `hardware/atm/cycle.py`.

7 · If something goes wrong

WHAT HAPPENS	WHAT TO DO
The card catches the edge of the reader's slot	Turn the reader's four height screws, all the same way: anticlockwise raises it (the card catches above the slot), clockwise lowers it. Beyond their travel, correct `reader_slot_bottom` and print the turret again.
Two cards come out together	Check the gate is fully down (its arms clicked under the steps): with the 1.0 mm gap, two cards (1.7 mm) can't pass.
The card stops short of the reader's end	Push the reader towards its slot with a shim behind it (0.5 to 9 mm, they stack; the long cable clip from 5 mm): the thinnest that lets the card in.
The card stops under the gate	Use gate 1.2.
The hopper moves, or the gate lifts	Check that all four of the hopper's fingers clicked under the rails, and both of the gate's arms under the steps on the pillars.
The carriage is stiff in its rails	Try `test_fit` with a larger `fit`, then print the channel floor and the carriage again.
A channel post won't go into the chassis, or comes out by itself	Make `chan_peg_d` 0.1 mm smaller (too tight) or larger (too loose), then print plate 10 again.
A post's split head won't go through the floor, breaks, or doesn't hold the floor down	Make `snap_barb_d` 0.2 mm smaller (won't go through, breaks) or larger (doesn't hold), then print plate 10 again.
The lever or the turret has play on its horn	Lower `horn_fit` (to 0.15, say) and print the lever and the turret again.
The lever swings outwards and jams	The servo turns the other way: its rest is 180° (see "Preparing the servos").
The turret isn't in line with the channel	Correct its rest angle in the calibration; if it is far off, move the horn by one spline.
A servo doesn't move	Check the wire from the batteries' – to the Pi's GND (pin 34), the signal pin (32 or 33) and the batteries.
A servo shakes, or the Pi resets	Power the servos from the batteries or their own supply of at least 3 A, never from the Pi's 5 V. Weak batteries make the servos shake: charge them. If a servo twitches while it holds still and the script printed a note about the hardware PWM, turn it on (see "Preparing the servos").
The ATM tips over when the phone is touched	Tape the feet to the counter, or put a weight behind the hopper.